

第三章 GNU と UNIXのコマンド

这章的核心只有一句话：

“如何在命令行里，把数据流动、加工、保存”

不是“记命令”，而是理解数据怎么走。

一、第三章的「主线逻辑」（非常重要）

你脑子里要有这一条流水线 📌

[输入]

↓

命令 (ls / cat / grep / sed / awk ...)

↓

管道 | / 重定向 > >>

↓

[输出 / 文件 / 下一个命令]

📌 整章 = 围绕“数据流”展开

3.1 在考什么（先给你一张“全景地图”）

3.1 本质考「人 → Shell → 命令 → 执行结果」这一整条链路，重点不是某个命令，而是：

命令是如何被 Shell 解释、查找、执行、控制、替换、记录、查看文档的

考试会从 8 个角度下手：

1. Shell 是什么 & 登录时发生了什么
2. 命令的基本输入与编辑
3. Shell 变量 vs 环境变量
4. PATH 与命令查找顺序
5. 命令的执行控制 (; && || () { })
6. 引用符 (' " \ \$() `)

7. 命令历史 (history / !)

8. man 手册 & section

二、3.1.1 Shell（一定会考）

1 Shell 的角色（一句话版）

Shell = 接收命令 → 找程序 → 启动进程 → 显示结果

📌 考试喜欢问：

- Shell **不是** 命令本身
 - Shell **不是** 内核
 - Shell 是 **用户和 Linux 之间的接口程序**
-

2 常见 Shell（只记“主流”）

Shell	说明	考试重点
sh	Bourne Shell	历史
bash	Bourne Again Shell	✅ Linux 默认
csh / tcsh	C 风格	次要
ksh	扩展 sh	次要
zsh	高功能	不常考

👉 一句口诀：

Linux 默认 = bash

3 登录 Shell 从哪来？

- 用户登录后启动的 Shell = **登录 Shell**
- 每个用户的 Shell 记录在：

/etc/passwd

示例：

linucuser:x:1000:1000::/home/linucuser:/bin/bash

👉 最后一列 = 登录 Shell

📌 必考点：

「用户的登录 Shell 由 /etc/passwd 决定」

4 Prompt（提示符）看懂就够

用户	Prompt
一般用户	\$
root	#

Prompt 的内容由环境变量 **PS1** 控制（知道即可，不深挖）

三、3.1.2 Shell 的基本操作（高频）

1 Tab 补全（几乎必考）

- 唯一候选 → **直接补全**
- 多个候选 →
 - Tab 一次：响铃 / 补公共部分
 - Tab 两次：列出所有候选

📌 出题方式：

输入到哪一步，按 Tab 会发生什么？

2 光标移动 & 编辑（表格直接背）

操作	含义
Ctrl+A	行首
Ctrl+E	行尾
Ctrl+D	删除光标字符 / 退出
Ctrl+H	向左删除
Ctrl+L	清屏
Ctrl+C	中断
Ctrl+Z	暂停
Ctrl+S / Q	锁屏 / 解锁

👉 这是“死记硬背表”，考试不会手软

四、3.1.3 Shell 变量 vs 环境变量（超核心）

1 一句话区分（一定要背）

Shell 变量：只在当前 Shell 有效
环境变量：对子 Shell 也有效

2 定义 / 参照规则（考试常阴人）

操作	正确?
VAR=linux	✓
\$VAR=linux	✗
echo VAR	✗
echo \$VAR	✓

🔑 口诀：

定义不用 \$，使用必须 \$

3 export 的真正含义

VAR=linux

export VAR

= 把 **Shell 变量** → **环境变量**

或者一行写完：

export VAR=linux

🔑 子 Shell 修改变量 → **不会影响父 Shell**

4 常见环境变量（会选项判断）

变量	含义
HOME	家目录
PATH	命令查找路径
USER	用户名
PWD	当前目录
LANG	语言
PS1	Prompt
HISTFILE	历史文件

五、3.1.4 PATH（出题最爱）

1 Shell 如何找命令？

顺序是：

| **PATH 从左 → 右 依次查找**

```
echo $PATH
```

2 为什么要 `./command`？

因为：

| **安全原因：PATH 默认不包含当前目录（.）**

所以当前目录下的程序：

```
./mycommand
```

🔴 这是 Linux 必考原理题，不是语法题

3 PATH 修改两种方式（会考对错）

✅ 正确追加：

```
PATH=$PATH:/opt/bin
```

❌ 覆盖原 PATH（危险）：

```
PATH=/opt/bin
```

六、3.1.5 命令执行控制（逻辑题）

符号	含义
;	不管成功失败都执行下一个
&&	成功才执行下一个
、	

例：

ls file && echo OK

ls file || echo NG

() vs **{ }** （容易混）

写法	特点
(cmd1; cmd2)	子 Shell
{ cmd1; cmd2; }	当前 Shell

 出题点：

子 Shell 中的变量不会回到父 Shell

七、3.1.6 引用符（重点中的重点）

1 三种引用的核心差异

写法	变量展开	命令替换
" "	✗ 不展开	✗
" "	✓ 展开	✓
` ` / \$()	执行命令	—

例子（考试爱用）：

DATE=date

echo \$DATE → date

echo "\$DATE" → date

echo '\$DATE' → \$DATE

echo \$(date) → 实际日期

🔴 推荐写法：

命令替换 `$()`，不要用反引号

2 反斜杠 \

- 转义下一个字符
 - Windows 日文环境中显示为 `¥`
👉 考试里把 ¥ 当作 \ 看
-

八、3.1.7 命令历史（必考）

常见用法

输入	含义
↑ / Ctrl+P	上一条
↓ / Ctrl+N	下一条
history	显示历史
!!	上一条
!5	第 5 条
!ls	最近以 ls 开头
!?pwd	包含 pwd

历史文件：

~/.bash_history

九、3.1.8 man 手册（送分题）

1 基本格式

man [选项] [章节] 命令

2 section（超爱考）

章节	内容
1	用户命令
5	配置文件
8	管理命令

例：

```
man 1 crontab # 命令
```

```
man 5 crontab # 配置文件格式
```

3 常用 man 选项

选项	含义
-a	所有章节
-f	完全匹配
-k	关键字搜索
-w	显示路径

4 less 操作（知道就行）

键	动作
j / k	上下
Space	下一页
b	上一页
/	向下搜索
?	向上搜索
q	退出

3.2 「パイプとリダイレクト」

一、先把“世界观”立起来：什么是「标准输入 / 输出」

Linux 有一个非常重要的设计思想：

一切皆文件，输入输出都是“流（stream）”

三个最基础的“通道”（一定要背）

编号	名称	英文	默认连接
0	标准输入	stdin	键盘
1	标准输出	stdout	屏幕
2	标准错误输出	stderr	屏幕

🔴 考试常考点：

- 0 / 1 / 2 = 文件描述符（file descriptor）
- 不是随便定的，是 OS 层面的编号

你可以在脑子里想成这样：



二、3.2.2 パイプ（pipe） | 把命令“接水管”

1 管道符号 | 是干嘛的？

把「前一个命令的标准输出」
接到「后一个命令的标准输入」

公式一定要刻在脑子里：

命令1 | 命令2

stdout(命令1) → stdin(命令2)

2 典型例子（你书上的）

ls | wc -l

拆解理解（非常重要）：

1. ls
→ 把文件名 输出到 stdout
2. wc -l
→ 没有参数时，会从 stdin 读数据
3. |
→ 把 ls 的 stdout 变成 wc 的 stdin

🔴 结果不是“ls 的结果”，而是 wc 的结果

3 为什么 `lsmod | less` 这么常见？

`lsmod | less`

这是真实运维现场级用法：

- `lsmod` 输出太多
- 直接显示 → 屏幕刷飞
- 用 `less`：
 - 一页一页看
 - 能 `/` 搜索
 - `q` 退出

🔴 考试会用它来考你：

| `less` 是“过滤器型命令”，可以接收 `stdin`

三、tee 命令 | 一份数据，两条路

1 tee 是干嘛的？

| 一边写文件，一边继续往下传

T 字形分叉，所以叫 `tee`

`ls -l | tee ls_log | wc -l`

数据流是：

`ls -l`

└─► 写入 `ls_log`

└─► 送给 `wc -l`

2 tee 的选项（只考一个）

选项	含义
-a	追加 (append) , 不覆盖

🔴 没有 `-a` 默认是覆盖

四、xargs | “把 stdin 变成参数”

这是 3.2 里最容易懵、但最重要的点之一。

1 xargs 在解决什么问题？

Linux 有一个限制：

单条命令的参数长度是有限的

所以这个会炸：

```
rm *
```

Argument list too long

2 xargs 的本质一句话

把标准输入读进来，拆成参数，再调用命令

```
echo * | xargs rm
```

等价于（概念上）：

```
rm file1 file2 file3 ...
```

3 和 find 的黄金搭配（考试常考）

```
find . -mtime +100 -type f | xargs rm
```

📌 你书里也明确写了：

下面这个在逻辑上等价：

```
find . -mtime +100 -type f -exec rm {} \;
```

👉 考试对比点：

- `xargs`：适合大量参数
- `-exec`：简单但可能慢

五、3.2.3 リダイレクト（重定向） | 改“流向”

1 输出重定向 `>` `/` `>>`

```
ls -l > filelist # 覆盖
```

```
ls -l >> filelist # 追加
```

📌 考点：

- 文件不存在 → 创建
- 存在 → `>` 覆盖，`>>` 追加

2 输入重定向 <

```
grep http < /etc/services > result.txt
```

意思是：

- stdin: 来自 `/etc/services`
- stdout: 写到 `result.txt`

3 Here Document <<

```
cat > sample.txt << EOF
```

LinuC

Linux

EOF

📌 重点不是写法，是概念：

在 shell 中临时构造 stdin

4 错误输出重定向（必考）

```
find / -name "*.tmp" 2> error.log
```

写法	含义
2>	stderr 覆盖写入
2>>	stderr 追加

5 合并 stdout + stderr（高频）

```
command > file 2>&1
```

顺序不能乱，这是考试坑点。

📌 含义拆解：

1. `file` : stdout → file
 2. `2>&1` : stderr → “当前 stdout 的去向”
-

六、最后给你一张「3.2 速记脑图（文字版）」

stdin(0) stdout(1) stderr(2)

管道 |

stdout → stdin

重定向

> stdout 覆盖

>> stdout 追加

< stdin 来源

2> stderr 覆盖

2>> stderr 追加

> file 2>&1 合并输出

tee

stdin → 文件 + stdout

xargs

stdin → 参数 → 命令

3.3 「テキスト処理フィルタ」

文本处理过滤器 =

从「标准输入」读文本 → 做一件很单一的处理 → 把结果丢到「标准输出」

🔴 关键词只有三个：

stdin → filter → stdout

所以它们几乎都要和管道 | 一起用。

一、先把 3.3 的命令「分门派」

你现在看到一堆命令，很乱是正常的。

我们先强行分 6 类 🙌

① 「看文件」系（不改内容）

命令	核心作用
cat	原样输出
nl	加行号
od	二进制视角看文件
head	看前面
tail	看后面

- 📌 共同点：
- 👉 只负责“显示”，不做内容变换

② 「截取」系（取一部分）

命令	取什么
cut	按字符 / 字段
head	前 N 行
tail	后 N 行

- 📌 考试高频点：
- cut -c = 字符位置
 - cut -d -f = 字段（delimiter）

③ 「横向 / 纵向拼接」

命令	方向
paste	横向合并
join	按字段关联合并

- 📌 一眼区分法：
- paste：不看内容，只拼行
 - join：必须有“共同字段”

④ 「空白 / 对齐」系（版式）

命令	干嘛
expand	tab → 空格
unexpand	空格 → tab
fmt	按列宽自动换行
pr	打印前排版

📌 这类命令：

👉 几乎不考复杂选项，只考“知道是干嘛的”

⑤ 「字符级变换」系（tr）

命令	作用
tr	字符替换 / 删除 / 压缩

📌 tr 的三板斧（必背）：

tr 'a-z' 'A-Z' # 大小写转换

tr -d '/' # 删除字符

tr -s ' ' # 连续空格压成1个

📌 tr 只能处理“字符”，不能处理“行”
(这是考试陷阱)

⑥ 「排序 / 去重 / 统计」系（超级重点）

命令	作用
sort	排序
uniq	去重 (△要先 sort)
wc	计数

📌 Linux 必考三连招：

sort file | uniq

sort file | uniq -c

ls | wc -l

二、几个必踩坑点（考试爱问）

🔥 1 `uniq` 不是万能去重

✗ 错误理解：

| `uniq` = 删除重复行

✓ 正确理解：

| `uniq` 只能处理“相邻重复行”

所以几乎一定要配 `sort`：

`sort file | uniq`

🔥 2 `sort` 默认不是“数字排序”

`sort test.txt` # 字符排序

`sort -n test.txt` # 数值排序

📌 看到 `-n`

👉 脑子里直接反射：数字

🔥 3 `cut` vs `tr` 的根本区别

命令	单位
<code>cut</code>	字段 / 字符位置
<code>tr</code>	单个字符

📌 只要题目说：

- “第几字段” → `cut`
 - “把 a 变成 A” → `tr`
-

🔥 4 `head / tail` 的隐藏点

`head -5 file`

`head -n 5 file` # 等价

tail -f logfile # 实时追日志

 `tail -f` = 日志监控代名词

三、考试用「极速判断表」

看到题目直接套👉

题目关键词	命令
显示前几行	head
显示后几行	tail
行号	nl
取字段	cut -d -f
转大写	tr
排序	sort
数字排序	sort -n
去重	uniq
统计行数	wc -l
tab↔space	expand / unexpand

3.4 《正規表現を使ったテキスト検索》

3.4 = 三层能力叠加

① 正则表达式（规则）



② grep（用正则“找”）



③ sed（用正则“改”）

考试与实务都是这个顺序，所以我们也按这个来。

一、3.4.1 正则表达式（这是“语言本身”）

正则不是命令，是一种描述字符串形状的语言

1 普通文字（最基础）

正则	含义
abc	就是字符串 abc

📌 考试点：
正则里，默认就是“字面匹配”

2 任意 1 个字符： .

正则	能匹配
a.c	abc / a2c / a-c

⚠ 注意：
. ≠ 空
. ≠ 任意长度
👉 只等于 1 个字符

3 字符类 [] （超级重要）

基本形式

[123] → 1 或 2 或 3

[a-z] → 任意小写字母

[0-9] → 任意数字

否定 (^ 在 [] 里)

[^abc] → 不是 a / b / c 的 1 个字符

⚠ 高频陷阱

位置	含义
^ 在 [] 里	否定
^ 在最外面	行头（下面讲）

4 行头 & 行尾（锚点 Anchor）

符号	含义
^	行头
\$	行尾

举例

^a → 行首是 a

a\$ → 行尾是 a

^\$ → 空行

🔴 考试特别爱问：

^\$ = 空行

5 重复（出现几次）

基本正则（grep 默认）

符号	含义
*	0 次以上

ab*z → az / abz / abbz

⚠ 关键点：

★ 可以是 **0 次**（这是很多人错的地方）

6 扩展正则（grep -E / egrep）

符号	含义
+	1 次以上
?	0 或 1 次
	或
{n}	n 次
{n,m}	n~m 次

🔴 考试判断法

看到这些 → 一定是 -E 或 egrep

二、3.4.2 grep：用正则“找”

1 grep 的本体结构（必背）

grep [选项] 正则 文件

2 grep 高频选项（表 3-32）

选项	含义	考试频率
-c	只显示匹配行数	★★★★
-i	忽略大小写	★★★★★
-n	显示行号	★★★★
-v	显示“不匹配”的行	★★★★★
-f	从文件读取正则	★★★
-E	使用扩展正则	★★★★★

3 grep 经典考试例子（你这本里全有）

忽略大小写

grep -i ab sample.txt

去掉注释行（超常考）

grep -v '^#' /etc/nsswitch.conf

🔴 这里的两个重点

- `^#` → 行头是 #
- 用 `' '` 包住 → 防止 shell 把 `#` 当注释

4 grep / egrep / fgrep 的关系（一定要分清）

命令	正则
grep	基本正则
grep -E / egrep	扩展正则
fgrep	不使用正则（纯字符串）

📌 例：

```
fgrep '*' file
```

→ 查的就是 `.*` 这两个字符

三、3.4.3 sed：用正则“改”

sed = 流编辑器

默认：不改原文件，只输出结果

1 sed 基本形态（三种）

sed '命令' 文件

sed -e '命令1' -e '命令2' 文件

sed -f 脚本 文件

2 sed 三大核心命令（表 3-35）

命令	含义
d	删除
s	替换
y	单字符替换

3 d：删除行

```
sed '1,5d' file.txt
```

👉 删除 1~5 行

📌 重点

- `sed` 不改原文件
 - `-i` 才会改（考试会问）
-

4 s：替换（最重要）

s/原/新/

s/原/新/g

写法	含义
无 g	每行只换第一个
有 g	全部替换

教材例子你已经见过：

```
sed 's/linux/LINUX/' linuxfile.txt
```

```
sed 's/linux/LINUX/g' linuxfile.txt
```

5 行范围 + s（考试杀手点）

```
sed '1,5s/^/>/' /etc/passwd
```

含义拆解：

- 1,5 → 行号范围
- ^ → 行头
- > → 加一个符号

👉 只在前 5 行加 >

6 y：字符对字符替换

```
sed 'y/ABC/123/' sample.txt
```

⚠ 规则：

- 左右字符数必须一样
 - 是“位置对应替换”
-

四、这一章你该怎么记（给你考试级记忆法）

🧠 正则速记

. 任意1字符

□ 选1个

[^] 排除

^\$ 行头 行尾

- 0次以上
- `{}` 扩展正则

grep 速记

grep -i 忽略大小写

grep -v 反选

grep -E 扩展正则

sed 速记

d 删除

s 替换

y 字符替换

-i 才改文件

3.5 「文件的基本编辑」

一、3.5 在整章里的定位（先站高一点看）

3.4 = 不动文件的文本处理

- grep / sed
- 📌 流式处理 (stdout)
- 📌 默认 不改原文件

3.5 = 真正“改文件内容”

- nano / vi / vim / emacs
- 📌 直接编辑文本文件
- 📌 配置文件、系统管理必备

📌 一句话总结

grep / sed 是“看、筛、临时改”，
vi / nano 是“真的动刀改文件”。

二、3.5.1 编辑器的基本（考试超爱问）

Linux 常见三大编辑器

编辑器	特点	考试态度
vi / vim	几乎所有系统都有	★★★★★ 必考
nano	操作简单，新手友好	★★
emacs	功能极强，程序员用	★（知道名字即可）

📌 重点句（考试会直接考）

- Unix / Linux 标准编辑器是 vi
- 多数 Linux 发行版实际用的是 Vim（Vi IMproved）
- Ubuntu 默认编辑器是 nano

三、EDITOR 环境变量（非常容易被忽略，但会考）

export EDITOR=/usr/bin/vim

这句是干嘛的？

- 指定 系统默认编辑器
- 影响：
 - crontab -e
 - visudo
 - 其他“自动打开编辑器”的命令

📌 考试理解方式

设置 EDITOR 后，
系统调用编辑器时就用你指定的那个。

📌 补一刀（高级但你能懂）

- 临时生效：当前 shell
- 永久生效：写进 ~/.bash_profile / ~/.bashrc

四、3.5.2 vi 编辑器的核心思想（这里是分水岭）

vi 只有一个核心概念：

模式切换（Mode）

两个最重要的模式

模式	作用
命令模式 (Command mode)	移动、删除、复制
输入模式 (Insert mode)	输入文字

👉 启动 vi 时默认是命令模式

五、输入模式（表 3-36，必背）

命令	含义（考试原意）
i	在光标前输入
a	在光标后输入
I	行首输入
A	行尾输入
o	在下一行插入
O	在上一行插入

📌 口诀（很重要）

- 小写：当前位置
- 大写：整行级别

退出输入模式：

Esc

六、光标移动（表 3-37，考试爱混）

基础移动（hjkl）

键	方向
h	左
l	右
k	上
j	下

👉 不是反人类，是为了不离开主键区

行 / 文件级移动

命令	含义
0	行首
\$	行尾
gg	文件开头
G	文件结尾
nG	第 n 行

🔴 重点：所有命令都能加数字

5j → 向下 5 行

20x → 删除 20 个字符

七、保存 / 退出 / 执行命令（表 3-38，必考）

vi 的“冒号命令”

命令	含义
:q	退出（未改）
:q!	强制退出
:w	保存
:wq	保存并退出
ZZ	保存并退出（等价 :wq）

在 vi 里跑 shell 命令（考点）

命令	含义
:!ls	不退出 vi 执行 ls
:r 文件	读入文件内容
:!命令	执行 shell 命令

 考试最爱问

vi 中可以不退出就执行 shell 命令吗？

 可以，用 `:!`

八、编辑命令（表 3-39，删除/复制/粘贴）

命令	含义
x	删除光标字符
X	删除前一个
dd	删除一行
dw	删除一个单词
yy	复制一行
p	向下粘贴
P	向上粘贴
r	替换一个字符

 删除 = 剪切

- dd 删除的行 = 自动进缓冲区
- 可以直接 `p` 粘回来

九、搜索 & 替换（表 3-40，和 3.4 正则强关联）

搜索

命令	方向
/pattern	向下找
?pattern	向上找
n	下一个
N	上一个

替换（考试高频）

命令	含义
:%s/A/B/	每行第一个 A → B
:%s/A/B/g	所有 A → B

📌 重点

- vi 的搜索支持 **正则表达式**
- 和你刚学的 3.4 完全联动

十、vi 设置（表 3-41）

命令	含义
:set nu	显示行号
:set nonu	关闭行号
:set ts=数字	Tab 宽度

📌 考试细节

- `:set` 是临时生效
- 永久生效 → `.vimrc`（去掉冒号）

十一、vimtutor（官方外挂答案）

vimtutor

- CentOS: `vim-enhanced`
- Ubuntu: `vim`

📌 考试态度

“通过 vimtutor 学习 vi 基本操作”

十二、3.5 的一句话终极总结（你背这个就够）

Linux 系统中，
vi 是标准文本编辑器，

通过命令模式和输入模式切换进行操作，
支持 移动、编辑、搜索、正则替换、执行 shell 命令，
是系统管理中不可或缺的技能。

第4章リポジトリとパッケージ管理

第4章只围绕这 5 个核心轴出题：

1. パッケージとは何か
2. Debian形式 / RPM形式的区别
3. dpkg vs apt vs apt-get vs apt-cache vs apt-file
4. リポジトリ / sources.list 的理解
5. 依存関係・競合関係

4.1 Debianパッケージ管理（完全版・考试向）

✓ STEP 1 | 先把「考试必须认识的概念」讲清楚（概念层）

这一层的目标：

你在考场看到日文名词时，脑子里立刻有“定义 + 作用 + 关键词”

① パッケージ（package）とは

日文原文核心：

実行プログラム、設定ファイル、ライブラリ、ドキュメントなどを
ひとまとめにしたもの

考试用理解（一定要这样记）：

- パッケージ =
👉 软件不是一个文件，而是一“包”文件集合
- 包里通常包含：
 - 実行ファイル（可执行程序）
 - ライブラリ（库）
 - 設定ファイル
 - マニュアル・ドキュメント

🔴 考试关键词

「ソフトウェアのインストールはパッケージ単位で行う」

看到这句话，直接联想到 **dpkg / apt / rpm / yum**

② パッケージ管理システムとは

日文定义要点：

- 管理以下事情的机制：
 - インストール
 - アンインストール
 - アップデート
- 并且：
 - 管理「どのパッケージが」
 - 管理「どこにインストールされているか」
 - 管理「競合・依存」

考试必背句型：

パッケージ管理システムは、
依存関係や競合関係を管理・監視する仕組み

🔴 为什么重要（考试常考）

- 如果依存关系崩坏 →
 - 👉 ソフトウェアが動作しない
 - 👉 システムが正常に動作しない

（这正好对应你图里的「ここが重要」框）

③ Debian形式 と RPM形式

这是 LinuC 超爱考的“分类题”

項目	Debian形式	RPM形式
代表系統	Debian / Ubuntu	RHEL / CentOS
包格式	.deb	.rpm
低層命令	dpkg	rpm
高層管理	apt / apt-get	yum
互換性	❌ なし	❌ なし

🔴 考試陷阱點

Debian形式とRPM形式には互換性がない

✓ 對

❌ 不能混裝

④ リポジトリ (repository)

定義（考試原文型）：

- パッケージを集積・管理している場所
- 文件系統目錄 **或** 網絡服務器

考試要點：

- 默認（標準）リポジトリ 是有的
- 如果標準倉庫沒有 → 可以追加 サードパーティリポジトリ
- 但：

- 標準以外のリポジトリは **自己責任**

🔴 例子（你圖里有）：

- CentOS 常用：**EPEL**

⑤ 依存關係 と 競合關係（这一块一定会考）

◆ 依存關係 (dependency)

定義：

- パッケージA が パッケージB を必要とする關係

👉 沒有 B，A 就不能用

◆ 競合関係 (conflict)

定义:

- 两个包试图安装 **同名文件** (但版本不同)
- 会「ぶつかる」

📌 考试高频句:

パッケージ管理システムは
依存関係や競合関係を監視し、問題があれば警告する

✅ STEP 2 | 再把「dpkg 的机制与用法」完整讲清 (命令层)

这一层目标:

看到 dpkg 选项, 你能马上判断 “是干嘛的”

4.1.2 dpkg コマンドを用いたパッケージ管理

① dpkg 的定位 (一定要清楚)

📌 考试定性结论:

- dpkg 是:
 - Debian 系
 - 低层 (低レベル)
 - 直接操作 `.deb` 文件
- 不会自动解决依存关系

👉 所以:

通常管理用 apt
但 **dpkg 仍然必须掌握**

② Debian パッケージ文件名结构 (必考)

例子:

tree_1.7.0-5_amd64.deb

编号	含义
①	パッケージ名
②	バージョン番号
③	Debianリビジョン番号
④	アーキテクチャ
⑤	拡張子 (.deb)

🔴 考试爱问方式：

- 「amd64 は何を表すか」 → アーキテクチャ
- 「.deb は何か」 → Debian形式パッケージ

③ dpkg 基本书式（直接记）

dpkg [オプション] アクション

④ dpkg 的主要アクション（考试核心）

◆ インストール

dpkg -i パッケージファイル名

- i = install
- `.deb` 文件路径

◆ アンインストール（两种，一定要分清）

命令	区别
-r / --remove	保留設定ファイル
-P / --purge	設定ファイルも 削除

🔴 考试陷阱题必出

「設定ファイルも含めて完全に削除したい」
✓ `--purge`

◆ 查询类（超高频）

目的	命令
已安装包列表	<code>dpkg -l</code>
查文件属于哪个包	<code>dpkg -S ファイル名</code>
查看包包含的文件	<code>dpkg -L パッケージ名</code>
查看包状态信息	<code>dpkg -s パッケージ名</code>

👉 这一组是 LinuxC 必背

⑤ dpkg-reconfigure（容易被忽略，但会考）

作用：

- 重新进行「対話的な設定」

`sudo dpkg-reconfigure postfix`

📌 考试点：

- 可以 随时 重新执行配置
 - 常用于 postfix、tzdata 等
-

📌 【4.1 必会 checklist】

你只要逐条自问，能不能立刻答：

- 什么是 パッケージ
- 什么是 パッケージ管理システム
- Debian形式 / RPM形式 区别
- 什么是 リポジトリ
- 什么是 依存関係
- 什么是 競合関係
- dpkg 是高层还是低层
- `.deb` 文件名各部分含义
- `-r` 和 `--purge` 的区别
- `dpkg -S` 是干嘛的
- `dpkg -L` 是干嘛的

- `dpkg -s` 是干嘛的
 - `dpkg-reconfigure` 的用途
-

4.2 APTによるパッケージ管理（超重点）

APT の核心思想（一句话）

リポジトリを利用して
依存関係を自動的に解決する

4.2.1 apt-get（考试主力）

1 apt-get 的地位（必考）

APT に含まれるコマンド
インストール・アップグレード・削除を行う

🔴 考试常见对比：

- `dpkg`：本地 `.deb`
 - `apt-get`：リポジトリ利用
-

2 apt-get update（100%会考）

！ 超重要原句

`apt-get update` は
システムを更新しない

👉 更新的是：

- パッケージ情報
- インデックス

🔴 判断题杀手锏：

- `apt-get update` = 系统升级 ❌
-

3 upgrade vs dist-upgrade（必考对比）

命令	特点
upgrade	不删除包
dist-upgrade	可能删除包（解决依赖）

4 autoremove（常考）

自動的にインストールされた
不要なパッケージを削除

sources.list（配置题）

1 配置文件位置（必背）

- `/etc/apt/sources.list`
- `/etc/apt/sources.list.d/`

2 deb 行的结构（考试必拆）

deb <http://archive.ubuntu.com/ubuntu> bionic main restricted

项	含义
deb	二进制包
URI	取得先
bionic	バージョン名
main	ソフトウェア区分

📌 **main / universe / restricted / multiverse**

👉 考试只要你知道 这是软件分类即可

4.2.2 apt-cache（查询专用）

📌 考试定位一句话：

パッケージ情報の検索・参照

子命令	功能
search	搜索
show	基本信息
showpkg	详细信息
depends	依存關係

👉 不要求已安装

4.2.3 apt-file（文件 → 包）

🔴 和 dpkg -S 的区别（超爱考）

命令	查对象
dpkg -S	已安装包
apt-file search	未安装也可

👉 所以 apt-file 必须先 update

考试终极对照表

需求	正确命令
更新包信息	apt-get update
安装包	apt-get install
查文件属于哪个包（已装）	dpkg -S
查文件属于哪个包（未装）	apt-file search
删除但留配置	dpkg -r / apt remove
完全删除	dpkg -P / apt purge

dpkg / apt / yum 速查对照表

目标：看到题目关键词 → 直接反射出命令

三者 在考试中的「身份定位」

工具	身份（考试原句级）	系统
dpkg	Debian形式の低レベルパッケージ管理	Debian / Ubuntu
apt / apt-get	リポジトリを利用する高レベル管理	Debian / Ubuntu
yum	RPM形式の高レベル管理	RHEL / CentOS

🔴 必考判断句

| dpkg は依存関係を自動解決しない（🔴）

最核心差异（考试最爱）

项目	dpkg	apt / apt-get	yum
使用仓库	🔴 不使用	🔴 使用	🔴 使用
依赖解决	🔴 不自动	🔴 自动	🔴 自动
操作对象	本地 .deb	仓库包	仓库包
层级	低级	高级	高级

安装 / 删除 / 完全删除（必背对照）

◆ 安装

需求	命令
安装本地 deb	dpkg -i xxx.deb
从仓库安装（Debian）	apt install xxx / apt-get install xxx
从仓库安装（RPM）	yum install xxx

◆ 删除（保留配置）

系统	命令	考试关键词
dpkg	dpkg -r xxx	設定ファイルを残す
apt	apt remove xxx	設定ファイル残す
yum	yum remove xxx	(配置由yum管理)

◆ 完全删除（配置也删）

🔴 考试陷阱

| purge 是 dpkg / apt 的概念，yum 没有

查询类（非常非常爱考）

1 已安装包列表

系统	命令
dpkg	dpkg -l
apt	apt list --installed
yum	yum list installed

2 文件属于哪个包（必考对比）

情况	正确命令
文件来自已安装包	dpkg -S /path
文件来自未安装包	apt-file search /path
RPM系统文件查包	yum provides /path

🔴 超经典考题

| /usr/bin/sudo はどのパッケージ？

👉 Debian: apt-file search /usr/bin/sudo

👉 RPM: yum provides /usr/bin/sudo

3 包里有哪些文件

系统	命令
dpkg	dpkg -l bash
apt	apt show bash
yum	yum repoquery -l bash（了解即可）

4 包的详细信息

系统	命令
dpkg	dpkg -s bash
apt	apt show bash
yum	yum info bash

五、更新 / 升级（必考概念题）

◆ Debian 系（apt）

命令	真正含义
apt update	更新包信息（不是系统）
apt upgrade	升级包（不删除）
apt full-upgrade	允许删除包

📌 100%会考句

| apt update はシステムを更新しない（○）

◆ RPM 系（yum）

命令	含义
yum check-update	检查更新
yum update	更新系统
yum upgrade	和 update 几乎等同（旧考点）

自动清理（高频）

系统	命令	含义
apt	apt autoremove	删除不再需要的依赖
yum	yum autoremove	同上

dpkg / apt / yum 的「一句话速记法」

- **dpkg:**
👉 「我有一个 deb 文件」
- **apt / apt-get:**
👉 「我不管依赖，系统帮我」
- **yum:**
👉 「我是 RedHat 世界的 apt」

考前 30 秒脑内默念版（终极）

dpkg 是低级
apt / yum 是高级
dpkg 不看仓库
apt / yum 看仓库
update ≠ upgrade
purge ≠ remove

4.3 RPMパッケージ管理

第4.3章 RPMパッケージ管理（考试版·完整整理）

目标：

- 用中文理解
- 用日语术语记忆
- 确保 LinuC 考试不会漏点

4.3.1 RPMパッケージ（RPM 包的基本概念）

① RPM 是什么（考试必背句）

日文原句核心：

RPM は Red Hat 社が開発したパッケージ管理システム

考试记忆版：

- RPM（Red Hat Package Manager）
- 由 Red Hat 开发
- 使用于：
 - RHEL
 - CentOS
 - Fedora
 - openSUSE / SUSE Linux Enterprise

📌 考点：

「APT ≠ RPM」
Debian系 → APT
Red Hat系 → RPM

② RPM 包文件名的结构（必考）

例子（你图里这条）：

bash-4.2.46-33.el7.x86_64.rpm

位置	含义（日语）	中文
①	パッケージ名	包名
②	バージョン番号	版本号
③	リリース番号	发布号
④	アーキテクチャ	架构
⑤	拡張子	扩展名（rpm）

📌 考试陷阱

- x86_64 是 architecture
- el7 属于 release

4.3.2 rpm コマンドの利用（rpm 命令）

⚠ 非常重要

rpm 是“低级命令”

- 不自动解决依赖
- 和 apt / yum 的定位不同

一、安装 / 升级模式（インストール・アップグレード）

① 安装（install）

rpm -i パッケージファイル名

- `-i` = install
- 从 **本地 rpm 文件** 安装

常见组合（图中强调）：

rpm -ivh xxx.rpm

选项	含义
-v	verbose（详细显示）
-h	hash（# 显示进度）

📌 考试点

`-ivh` 是典型写法

② 升级（upgrade / freshen）

命令	行为差异（必考）
-U	若未安装 → 当作新安装
-F	仅升级已安装的包

📌 一句话记忆

- `-U` = 有就升，没有就装
- `-F` = 只升不装

二、依赖相关（超常考点）

依赖检查失败的情况（你图里 mod_ssl 例子）

error: Failed dependencies

原因：

- rpm 不会自动解决依赖

强制忽略依赖（危险但必考）

rpm -ivh --nodeps xxx.rpm

选项	含义
--nodeps	忽略依赖关系

📌 考试固定说法

依存関係を無視してインストールする

⚠ 实务警告（考试不考）：

- 可能导致系统异常

三、卸载（アンインストール）

rpm -e パッケージ名

选项	含义
-e	erase

忽略依赖卸载：

rpm -e --nodeps パッケージ名

📌 考试点

- 卸载用的是 包名，不是文件名

四、查询模式（照会モード）★重中之重

① 查询是否安装 & 搜索

rpm -qa

- 显示 **所有已安装包**

结合 grep（图中例）：

```
rpm -qa | grep vim
```

② 查询包信息

```
rpm -qi パッケージ名
```

- 显示：
 - Name
 - Version
 - Architecture
 - Install Date
 - Summary 等

 考试高频

```
-qi = information
```

③ 查询文件属于哪个包（超爱考）

```
rpm -qf /bin/bash
```

选项	含义
-qf	file → package

 对比记忆

- rpm: `-qf`
- apt: `apt-file search`

④ 安装前查看 rpm 内容

```
rpm -qlp xxx.rpm
```

选项	含义
-l	list
-p	package file

📌 考试点

- 未安装也能看内容
-

⑤ 查询依赖关系

rpm -qR パッケージ名

- 显示该包依赖哪些文件 / 库
-

五、签名确认（パッケージの署名確認）

rpm --checksig xxx.rpm

或：

rpm -K xxx.rpm

📌 考试句型

電子署名を確認する

4.3 这一章你需要“会到什么程度”？

如果你能做到👉，4.3 就是稳的：

- 看到 `bash-4.2.46-33.el7.x86_64.rpm`
→ 能拆结构
 - 分得清：
 - `-i / -U / -F`
 - 记得：
 - rpm 不自动解决依赖
 - 一看到题目问：
 - “哪个命令能知道文件属于哪个包”
→ `rpm -qf`
-

第4.4章 yumコマンドによるパッケージ管理（考试版・完整）

适用对象：CentOS / Fedora（Red Hat 系）

定位一句话：

yum = RPM 系的 APT（自动处理依赖）

4.4.1 yum 是什么（先把定位钉死）

教材原意：

- CentOS / Fedora 中
- APT 相当工具是 YUM（Yellow dog Updater, Modified）

🔴 考试对比记忆

- Debian 系：apt / apt-get
- Red Hat 系：yum
- rpm ≠ yum
 - rpm：低级工具，不自动解决依赖
 - yum：高级工具，自动解决依赖

4.4.2 yum 的配置文件（必考路径）

① 主配置文件

/etc/yum.conf

你图里的关键项（考试只认这些）：

项目	含义
cachedir	缓存目录
logfile	yum 日志文件
gpgcheck	是否检查 GPG 签名
plugins	是否启用插件

🔴 考试爱问

yum 的日志文件在哪里？

👉 /var/log/yum.log

② 仓库配置目录（超高频）

/etc/yum.repos.d/

- 里面是 *.repo 文件
- 一个 repo 文件 = 一个仓库定义

🔴 和 APT 的对照（只记，不写）

- apt: /etc/apt/sources.list
 - yum: /etc/yum.repos.d/*.repo
-

4.4.3 yum 基本命令结构

yum サブコマンド [オプション] [パッケージ名]

4.4.4 yum 的主要子命令（表4-7，全是考点）

一、更新相关（アップデート）

① 检查可更新包（不更新）

yum check-update

🔴 考试点

- 只显示
 - 不安装
 - 类似 apt 的「更新前查看」
-

② 更新全部包

yum update

- 系统中**所有可更新包**都会更新
 - **依赖包也会一起更新**
-

③ 更新指定包

yum update openssl

🔴 关键词

指定したパッケージをアップデートする

二、安装 / 卸载（インストール・アンインストール）

① 安装包

`yum install ruby`

特点（必背）：

- 从网络仓库获取
 - 自动解决依赖
-

② 卸载包

`yum remove emacs`

和 rpm 的关键差异

- `yum remove`：会处理依赖
 - `rpm -e`：不管依赖（除非你手动）
-

三、包信息查询（照会系，重点）

① 显示包信息（是否安装都可）

`yum info bash`

考试句型

| インストールされていないパッケージの情報も表示できる

② 列出包列表

`yum list`

输出包括：

- Installed Packages
- Available Packages

高频细分

`yum list installed`

`yum list available`

（教材里没展开，但考试默认你知道）

③ 搜索包（关键字）

yum search ruby

📌 search vs search all

- `search` : 包名 + 概要
 - `search all` : 包名 + 说明文等全部
-

四、仓库操作（repo）

查看仓库列表

yum repolist

📌 考试点

リポジトリ一覧を表示する

4.4.5 包组（グループ）操作 ★必考

① 查看包组列表

yum groups list

常见组（你图里）：

- Minimal Install
 - GNOME Desktop
 - Development Tools
-

② 安装包组

yum groups install "GNOME Desktop"

📌 考试关键词

パッケージグループ単位でインストール

4.4.6 自动确认（-y 选项）

yum -y update

含义：

- 自动回答 **yes**
- 不进行交互确认

🔴 考试句型

確認を省略する

4.4.7 dnf (补充说明, 轻考点)

教材原文要点:

- CentOS 8 起
- 用 **dnf** 取代 yum
- 基本用法与 yum 几乎相同

🔴 考试级别理解

dnf = yum 的后继工具

4.4 你现在“合格线”是什么?

如果你能做到👉, 4.4 在考试里是**送分章**:

- 一看到题目问:
 - “哪个命令只显示可更新包?”
→ `yum check-update`
 - 问:
 - “yum 的仓库配置文件在哪里?”
→ `/etc/yum.repos.d/`
 - 问:
 - “按组安装 GNOME 桌面?”
→ `yum groups install`
 - 问:
 - “自动回答 yes?”
→ `-y`
-

APT / YUM / RPM 易混点终极整理 (考试向)

一、先一句话定性（一定要背）

工具	一句话定位
rpm	最底层的包管理工具（不自动解决依赖）
yum	RPM 系的高级工具（自动解决依赖）
apt / apt-get	Debian 系的高级工具（自动解决依赖）

📌 考试第一判断点

| 自动解决依赖？

- ❌ rpm
- ✅ yum / apt

二、系统归属（必考送分）

系列	工具
Debian / Ubuntu	apt / apt-get
CentOS / Fedora / RHEL	yum + rpm

⚠️ 考试陷阱

- 题目写 **CentOS** → 直接排除 apt
- 题目写 **Debian** → 直接排除 yum / rpm

三、层级关系（最容易混）

Debian系： apt



dpkg

RedHat系： yum



rpm

📌 关键理解

- apt 底层用的是 dpkg
- yum 底层用的是 rpm
- dpkg / rpm 都是 “低级工具”

四、安装 / 卸载的本质差异（超高频）

1 安装

工具	命令	依赖
rpm	rpm -ivh xxx.rpm	✗ 不处理
yum	yum install xxx	✓ 自动
apt	apt install xxx	✓ 自动

🔴 考试经典问法

依存関係を自動的に解決するツールはどれ？

👉 yum / apt

2 卸载

工具	命令	依赖
rpm	rpm -e xxx	✗
yum	yum remove xxx	✓
apt	apt remove xxx	✓

⚠ rpm 卸载可能留下 “孤儿依赖”，考试就爱考这个风险

五、仓库配置文件（一定要区分）

◆ APT（Debian系）

/etc/apt/sources.list

或

/etc/apt/sources.list.d/

◆ YUM (RedHat系)

/etc/yum.conf

/etc/yum.repos.d/*.repo

📌 考试最爱问

| yum 的仓库配置文件放在哪?

👉 /etc/yum.repos.d/

六、更新机制对比 (最容易选错)

APT

apt update ← 更新 “包信息”

apt upgrade ← 更新 “已安装包”

YUM

yum check-update ← 只显示可更新包

yum update ← 实际更新

📌 考试陷阱对照

- “显示可更新列表”
→ yum check-update
- “实际更新系统”
→ yum update

七、包文件 vs 仓库 (核心理解)

工具	安装来源
rpm	本地 .rpm 文件
yum	仓库 (repo)
apt	仓库 (repo)

📌 考试句型

| rpm パッケージはファイル単位で管理
yum はリポジトリ単位で管理

八、rpm 特有考点（不要混进 yum）

rpm 能做、yum 不强调的

功能	命令
查询已安装	rpm -qa
查询包信息	rpm -qi
校验签名	rpm --checksig

🔴 看到 `--checksig`
→ 直接选 rpm

九、dnf 是什么（轻考点）

项目	内容
出现	CentOS 8
关系	yum 的后继
用法	基本与 yum 相同

🔴 考试答案套路
dnf は yum とほぼ同じ使い方

十、终极防混口诀（你一定用得上）

rpm 不管依赖，yum / apt 才是王道
Debian 用 apt，RedHat 用 yum
yum 看 repo，rpm 看文件

第5章 ハードウェア / ディスク / パーティション / ファイルシステム

这一章在 LinuC 里是：
“Linux 是如何把一块真实的硬件，变成你能用的文件”的全过程

整章一句话主线（先记这个）

硬件 → 设备 → 分区 → 文件系统 → 挂载 → 使用

考试里的所有题，都绕不开这条链。

5.1 硬件的基础知识与设置 —— 你必须建立的 3 个层次

我先给你三层结构，后面所有内容都会往这里归位：

【固件层】 BIOS / UEFI

↓

【内核层】 Linux Kernel（驱动、/proc、/sys、模块）

↓

【用户层】 命令、文件、应用程序

记住这三层，你不会迷路。

一、CPU / 内存 / 存储 —— 不是“硬件介绍”，而是Linux 视角

1 CPU（中央处理器）

你要从 Linux 的角度理解 CPU：

- CPU ≠ 单一东西
- Linux 关心的是：
 - 架构（Architecture）
 - 核心数 / 线程
 - 是否支持 64bit
 - 是否支持虚拟化、指令集

🔴 考试必记的架构关键词

- x86 / IA-32 → 32 位
- x86_64 → 64 位（现在主流）
- ARM → 嵌入式 / 手机 / 云服务器

👉 为什么后面安装包、内核、系统都会提 architecture？

因为：

二进制程序只能跑在匹配的 CPU 架构上

2 内存（Memory / RAM）

一句话定性：

内存 = 临时工作台，断电就清空

Linux 里内存的关键点不是“多大”，而是：

- 谁在用？
- 用到什么程度？
- 不够了怎么办？

🔪 你后面会反复遇到：

- `/proc/meminfo`
- `free`
- `swap`

👉 重要理解（考试+实务）

- Linux 会尽量吃满内存（cache/buffer）
 - 内存用得多 ≠ 有问题
 - Swap 是“兜底方案”，不是内存替代品
-

3 存储（Storage）

这里是后面分区、文件系统的入口。

Linux 的基本认知：

- 存储是 持久化
- 内核把存储设备抽象成 块设备（block device）
- 块设备最终会变成：
- `/dev/sda`
- `/dev/sda1`
- `/dev/nvme0n1`

🔪 接口/规格（你要能区分“定位”）

名称	定位
SATA	PC/普通服务器主流
SAS	企业服务器（快+贵）
SCSI	老规范，概念仍在
USB	外接、热插拔

👉 关键不是速度，是“用途区分”（考试常考）

二、输入设备 / 扩展卡 —— 为什么 Linux 不在乎你插了什么？

输入设备（键盘、鼠标）

- 对 Linux 来说：只是设备
- 不重要的是“外形”
- 重要的是：
 - 是字符设备？
 - 是块设备？
 - 有没有驱动？

扩展卡（NIC / RAID / HBA）

🔴 考试会暗示的关键点

- 扩展卡 ≠ 自动可用
 - 需要：
 - BIOS/UEFI 识别
 - 内核驱动支持
 - 模块加载
-

三、BIOS / UEFI —— Linux 之前发生的事

这是很多人模糊、但考试非常爱考的部分。

1 BIOS / UEFI 是什么？

一句话版：

BIOS / UEFI = 开机后第一个“操作系统之前的系统”

它负责：

- 上电自检（POST）
- 初始化硬件
- 决定 **从哪启动**
- 把控制权交给 OS

 UEFI vs BIOS（必考对比）

项目	BIOS	UEFI
界面	文本	GUI
启动盘限制	2TB	基本无
分区	MBR	GPT
现代服务器	✗	✓

 **考试里看到“大容量磁盘 / GPT / Secure Boot” → 直接联想到 UEFI**

2 启动顺序（Boot Order）

这是 **服务器运维常识**：

- 光盘
- 硬盘
- 网络（PXE）

 为什么要记？

- 装系统
- 救援
- 无人值守服务器

四、Linux 如何“看到”硬件 —— 这是本章灵魂

1 /dev —— 设备文件

Linux 的世界观：

硬件 ≈ 文件

- `/dev/sda`
- `/dev/tty`

- `/dev/null`

📌 重点：

- `/dev` 不是随便放文件
 - 由 **udev** 动态生成
-

2 `/proc` —— 内核状态窗口

| `/proc` = 内核的“实时状态投影”

你不是在看文件，你是在**看内核此刻的状态**。

📌 必背文件（考试常客）：

- `/proc/cpuinfo`
 - `/proc/meminfo`
 - `/proc/interrupts`
 - `/proc/swaps`
-

3 `/sys` —— 设备模型的结构化视图

- 比 `/proc` 更“设备导向”
- udev 的信息来源之一

📌 典型考点：

`/sys/block/sda/sda1/size`

五、设备驱动 & 内核模块（5.1 的高潮）

1 驱动是什么？

| 驱动 = 内核中懂硬件语言的翻译官

Linux 中：

- 驱动 = 内核模块（module）
-

2 三个你必须形成肌肉记忆的命令

命令	作用
lsmod	看当前加载的模块
modprobe	加载模块（自动处理依赖）
rmmod	卸载模块

 考试陷阱

- `insmod` ≠ 推荐
- **modprobe 才是正确答案**

六、udev —— 为什么你不用手动创建设备文件

这是 Linux 现代化的关键机制。

一句话理解：

| **硬件变化 → 内核感知 → udev 创建设备文件**

流程：

硬件插入

- 内核发现
- /sys 生成信息
- udev 规则匹配
- /dev 下生成节点

 考点：

- udev 规则路径：
- `/etc/udev/rules.d/`
- 可以给设备 **自定义名字**

现在你已经完成了什么？

你已经具备：

-  硬件 → 内核 → 用户空间的完整认知
-  为什么 Linux 不直接“操作硬件”
-  为什么会有 `/dev` `/proc` `/sys`
-  为什么驱动加载失败 = 设备不可用

MBR / GPT / 主分区 / 逻辑分区

MBR / GPT 是“分区表格式”；

主分区 / 扩展分区 / 逻辑分区 是“MBR 时代为了解决数量限制搞出来的结构”；

GPT 时代基本只剩“普通分区 + EFI 系统分区”，逻辑分区这套直接退役。

👉 **GPT 根本没有“主 / 逻辑”这种概念**

这是第一条**生死线**。

一、先把“层级关系”搞清楚（非常重要）

硬盘相关的层级是这样的（从外到内）：

硬盘

└─ 分区表（MBR 或 GPT）

└─ 分区（primary / extended / logical / GPT partition）

└─ 文件系统（ext4, xfs, swap...）

👉 **MBR / GPT 决定“你能怎么分区”**

主 / 扩展 / 逻辑 决定“MBR 下分区怎么组织”

二、MBR 是什么？（老规矩）

MBR（Master Boot Record） 是一种老式分区表格式。

MBR 的三大硬性限制（考试必考）

1. 最大支持 2TB 硬盘
2. 最多只能有 4 个“基本分区（主分区）”
3. 启动信息 + 分区表都挤在磁盘最前面

👉 问题来了：

如果我想要 5 个分区怎么办？

三、主分区（Primary Partition）

主分区 = MBR 允许的“直接分区”

- 在 MBR 下：

- 最多 4 个
- 设备名: `/dev/sda1 ~ /dev/sda4`
- 主分区里可以直接：
 - 放文件系统
 - 挂载
 - 启动（某些分区）

🔴 到这一步，MBR 已经用完名额了

四、扩展分区（Extended Partition）是“壳”

为了解决「只能 4 个主分区」的问题，MBR 引入了一个很丑但实用的补丁方案：

扩展分区的本质

扩展分区本身不是用来放数据的，而是一个“容器 / 外壳”

- 扩展分区：
 - 本质上仍然算 1 个主分区
 - 只能有 1 个
 - 里面不能直接放文件系统

示意：

`/dev/sda3` ← 扩展分区（壳）

五、逻辑分区（Logical Partition）才是真正干活的

逻辑分区 = 放在扩展分区里面的“真正可用分区”

- 特点：
 - 只能存在于扩展分区内部
 - 数量几乎不限
 - 从 `/dev/sda5` 开始编号
 - 不管前面用了几个主分区

示例（你书里的经典结构）：

`/dev/sda1` 主分区

`/dev/sda2` 主分区

/dev/sda3 扩展分区

└─ /dev/sda5 逻辑分区

└─ /dev/sda6 逻辑分区

/dev/sda4 主分区

🔴 记忆点（考试杀手锏）

逻辑分区 一定 $\geq$ sda5

六、MBR 整体结构一句话总结

- MBR:
- 最多 4 个主分区
 - 其中 1 个可以变成扩展分区
 - 扩展分区里可以切很多逻辑分区

这就是你教材 5.2.1 那一整页的核心。

七、GPT 是什么？（新时代）

GPT（GUID Partition Table）是为了解决 MBR 全部缺点而生的。

GPT 直接把 MBR 那套推翻了

对比项	MBR	GPT
最大磁盘	2TB	几乎无限
分区数量	4（靠扩展硬撑）	默认 128 个
主 / 扩展 / 逻辑	有	没有
可靠性	差	高（有备份表）
启动方式	BIOS	UEFI

👉 GPT 里只有一种概念：分区

八、GPT 下的分区世界（非常干净）

在 GPT + UEFI 系统中：

- 不存在：
 - 扩展分区

- 逻辑分区
- 只有：
 - 普通分区
 - **EFI System Partition (ESP)**

EFI 系统分区 (ESP)

- 用途：**存放启动程序**
- 常见：
 - FAT32
 - 约 100~500MB
- 示例：
 - `/dev/sda1` → EFI
 - `/dev/sda2` → `/`
 - `/dev/sda3` → `/home`

考试关键句

GPT 系统中不使用扩展分区、逻辑分区

九、把所有概念压缩成一张“脑内表”

① MBR (老)

- 分区表：MBR
- 分区类型：
 - 主分区（最多 4）
 - 扩展分区（壳，最多 1）
 - 逻辑分区（扩展里，从 sda5 开始）

② GPT (新)

- 分区表：GPT
- 分区类型：
 - 普通分区
 - EFI 系统分区
-  没有逻辑分区

十、考试最爱问的 5 个判断点（你直接背）

1. 逻辑分区一定在扩展分区里面 ✓
2. 扩展分区本身不能直接存文件系统 ✓
3. `/dev/sda5` 一定是逻辑分区（MBR） ✓
4. GPT 不存在逻辑分区 ✓
5. UEFI + GPT 需要 EFI 系统分区 ✓

给你一套“秒选答案”的口诀

👉 判断是不是 MBR：

- 看到：
 - 主分区
 - 扩展分区
 - 逻辑分区
 - 4 个限制
- 👉 直接选 MBR

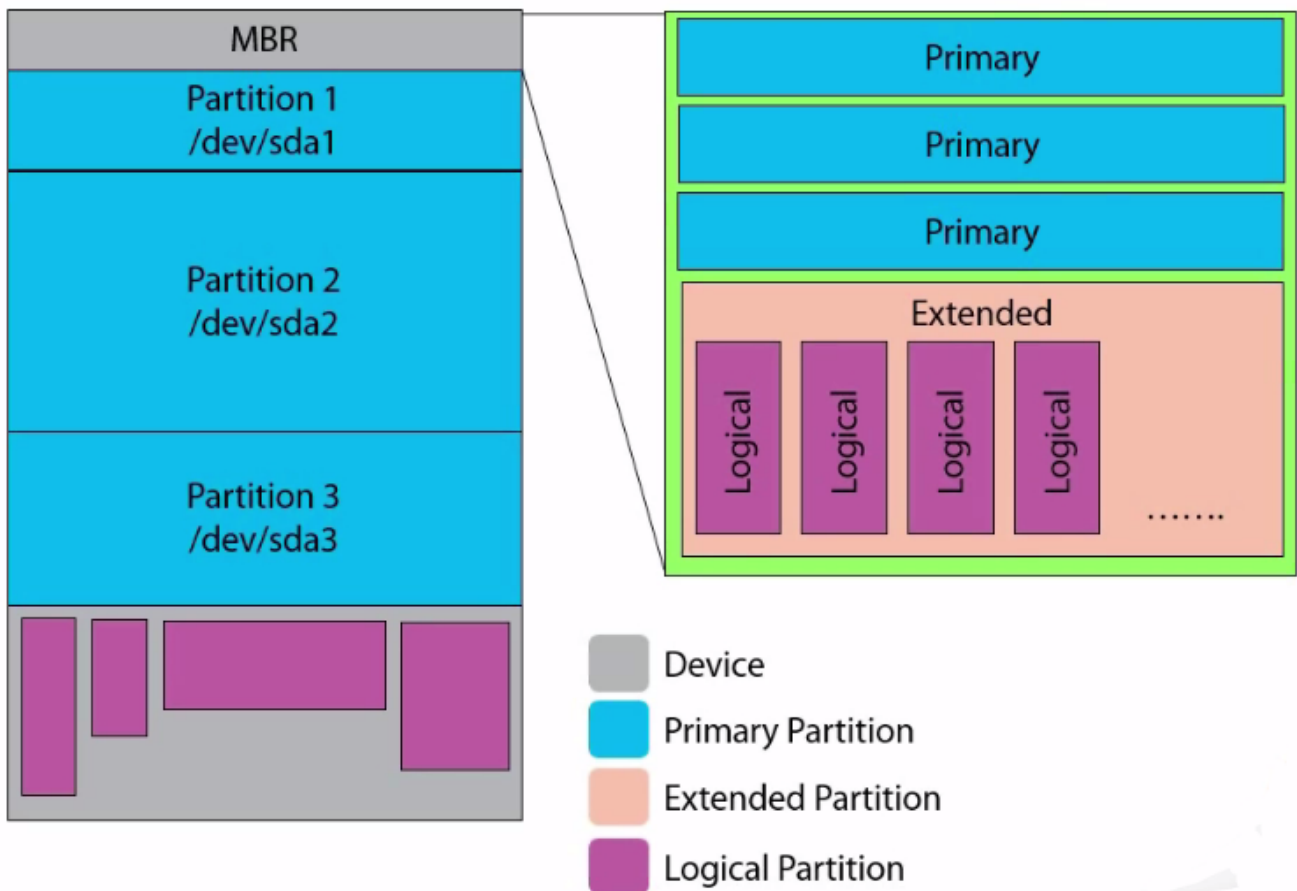
👉 判断是不是 GPT：

- 看到：
 - 大容量磁盘
 - UEFI
 - 分区很多
 - 没提主/逻辑
- 👉 直接选 GPT

5.2.1 パーティションの種類（分区的种类）

这一小节是 整章最容易乱的地方，但逻辑其实很简单。

MBR Partition Scheme



① 什么是 Partition（分区）

- 分区 = 把一块物理磁盘切成多个逻辑区域
- 每个分区：
 - 可以 单独格式化
 - 可以 挂载到不同目录
 - 可以 用不同文件系统

✖ 注意：

👉 分区 ≠ 文件系统（这是超高频混点）

② BIOS vs UEFI 决定了你用哪一套分区规则

启动方式	分区表	特点
BIOS	MBR (dos)	老体系
UEFI	GPT	新体系

③ MBR (= DOS 分区表) ——教材重点 + 易考

MBR 的三种分区 (这是 5.2 的核心)

1 基本分区 (Primary Partition / 基本パーティション)

- 最多 4 个
- 设备名：
 - `/dev/sda1` ~ `/dev/sda4`
- 可以直接放文件系统

2 扩展分区 (Extended Partition / 拡張パーティション)

- 本质：
 - 占用 1 个基本分区名额
 - 自己不能放文件系统
- 作用：
 - 只是一个“容器”

📌 考点句式：

扩展分区中**不存放文件系统**，而是存放逻辑分区

3 逻辑分区 (Logical Partition / 論理パーティション)

- 只能存在于扩展分区内部
- 编号规则 (必考)：
 - 从 `/dev/sda5` 开始
- 数量几乎无限

📌 一眼看懂教材图 5-4

`/dev/sda1` 基本

`/dev/sda2` 基本

`/dev/sda3` 扩展

└— `/dev/sda5` 逻辑

└─ /dev/sda6 逻辑
└─ /dev/sda7 逻辑
/dev/sda4 基本

④ GPT（UEFI 用）——规则完全不一样

GPT 的特点（考试直接选）

-  没有扩展分区 / 逻辑分区
-  全部都是“基本分区”
-  最多 128 个分区
-  支持 2TB 以上磁盘

EFI 系统分区（ESP）

- UEFI 必须
- 用途：
 - 存放 **引导程序**
- 一般格式：
 - `FAT32`
- 名字：
 - **EFI System Partition**

 教材原话关键点：

| GPT では拡張パーティション、論理パーティションは使用しない

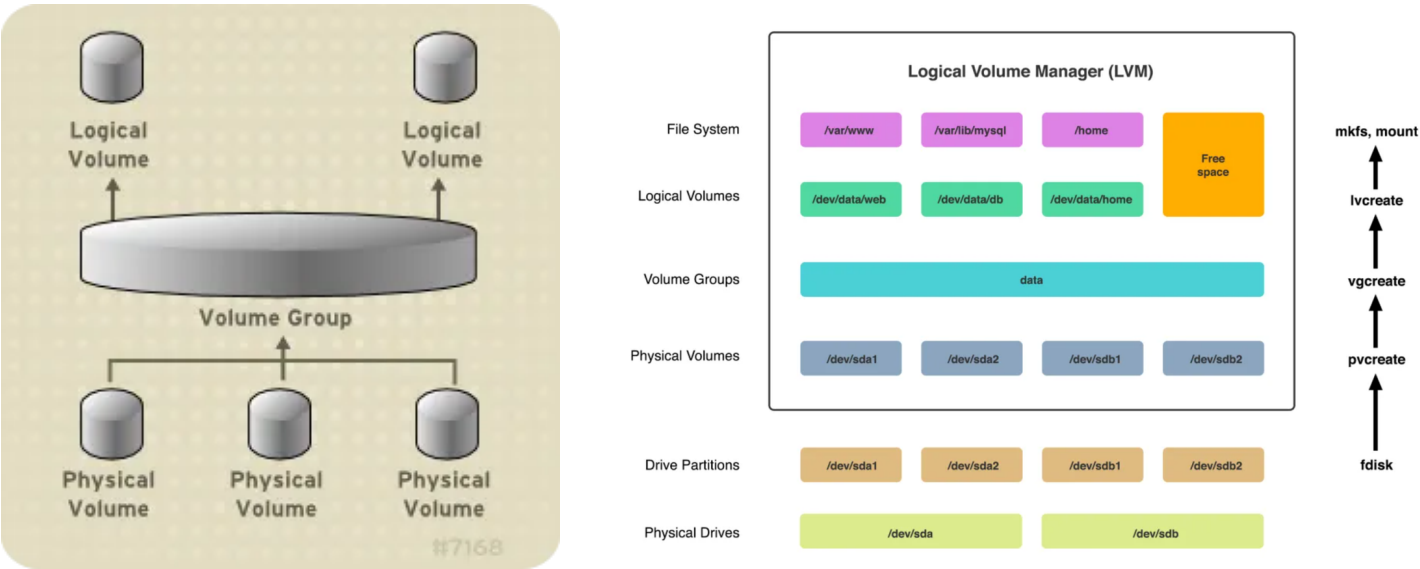
5.2.2 LVM（逻辑卷管理）——不是分区，但考试爱考

先一句定位

| LVM 是“分区之上的一层抽象”

三个核心名词（必背）

缩写	日文	中文
PV	Physical Volume	物理卷（磁盘/分区）
VG	Volume Group	卷组（虚拟磁盘）
LV	Logical Volume	逻辑卷（像分区一样用）



LVM 解决了什么问题（教材 bullet 全是考点）

- ✗ 普通分区不好改大小
- ✗ 分区不能跨磁盘
- ✗ 不能灵活扩展
- ✓ LVM 全部解决

📌 重点句：

ボリュームグループは仮想的なディスク
 論理ボリュームは仮想的なパーティション

5.2.3 デバイスファイル（设备文件）——接下来 5.3 的铺垫

Linux 的核心思想

一切皆文件

磁盘设备文件（Block Device）

- `/dev/sda`：第 1 块磁盘

- `/dev/sdb`：第 2 块磁盘
- `/dev/sda1`：第 1 块磁盘的第 1 个分区

用什么看？

lsblk

🔴 教材强调：

ディスクやパーティションはブロックデバイス

5.2.4 分区管理命令（考试只考“能区分”）

命令	支持	特点
fdisk	MBR（新版本也支持 GPT）	传统
gdisk	GPT	专用
parted	MBR + GPT	立即生效

🔴 超级易考陷阱

`fdisk / gdisk`：要 `w` 才写盘
`parted`：立即生效

🧠 5.2 到这里，你应该能回答这些题（自检）

你现在应该 不用看书就能回答：

1. MBR 为什么最多只能 4 个基本分区？
2. `/dev/sda5` 一定是什么分区？
3. GPT 下还有没有逻辑分区？
4. LVM 的 VG 和 LV 分别像什么？
5. 为什么 UEFI 必须有 ESP？

5.3 ファイルシステムの创建与管理

第 5 章逻辑是这样的：

硬件 → 磁盘 → 分区 → 文件系统 → 怎么让它“用起来”

↑

这里就是 5.3

5.3 的本质一句话：

👉 文件系统只有「被 mount 到目录树上」，用户和程序才能访问

二、什么是「マウント」？（考试用一句话定义）

マウント (mount) とは：

👉 把一个文件系统，连接到 Linux 的目录树上的某个目录（マウントポイント）

🔪 注意考试关键词：

- ❌ 不是“复制文件”
 - ❌ 不是“移动数据”
 - ✅ 是“把文件系统接到目录树上”
-

三、你图里的核心示意（我直接用语言还原）

1 mount 前（DVD-ROM 还没接上）

- `/media/dvdrom`：只是一个空目录
 - DVD 里的 `data`：系统看不到
-

2 mount 后

DVD-ROM ——mount——▶ `/media/dvdrom`

结果是：

`/media/dvdrom/data`

🔪 考试爱问的点：

DVD 里的 `data` 目录

👉 不是被复制到 `/media` 下

👉 而是通过 mount “显示出来”

四、为什么教材反复强调 `/media`？

教材里的 ★重点框 说的是：

`/media` ディレクトリ以下は

翻成考试语言就是：

- `/media`
👉 U盘 / DVD / 外接磁盘 的标准挂载点
(但不是“只能”，只是“常用 & 规范”)

五、mount 命令（5.3.9 的绝对核心）

① mount 的两种基本用法（必背）

✅ 用法 1：只敲 `mount`

`$ mount`

👉 显示当前已挂载的所有文件系统

🔴 常见输出里会看到：

- `proc`
- `sysfs`
- `tmpfs`
- `devtmpfs`

⚠ 考试陷阱：

这些是虚拟文件系统，不是磁盘分区。

✅ 用法 2：指定设备 + 挂载点

`mount -t ext4 /dev/sdb3 /data`

逐项拆解（这一步你一定要会）：

部分	含义
<code>-t ext4</code>	指定文件系统类型
<code>/dev/sdb3</code>	分区设备
<code>/data</code>	挂载点（必须是目录）

② mount 的主要选项（表 5-15）

- ◆ `-a`（超常考）

mount -a

👉 把 `/etc/fstab` 里写的文件系统全部挂载

⚠ 但：

- `noauto` 的 不会挂

-
- ◆ `-t` 文件系统名

`-t ext4`

`-t xfs`

👉 显式指定文件系统类型

-
- ◆ `-o` オプション

`-o rw`

`-o ro`

👉 指定挂载方式（读写 / 只读等）

六、`/etc/fstab` 的位置（考试必考关联点）

教材这句很关键：

デバイス名またはマウントポイントを省略すると
`/etc/fstab` が参照される

意思是：

如果你写：

mount /data

👉 系统会去 `/etc/fstab` 里找：

- 哪个设备
- 什么文件系统

- 用什么选项

📌 结论：

- `mount` ≠ 随使用
 - 长期挂载 = 靠 `/etc/fstab`
-

七、umount（アンマウント）——和 mount 一样重要

① 基本用法（两种都对）

`umount /data`

或

`umount /dev/sdb3`

👉 卸载的是“同一个挂载关系”

② umount 的主要选项（表 5-16）

◆ `-a`

`umount -a`

👉 卸载 `/etc/mtab` 中记录的所有文件系统

(`mtab` = 当前实际挂载状态)

◆ `-t ファイルシステム名`

`umount -at xfs`

👉 卸载所有 xfs 文件系统

📌 这是考试非常爱考的组合。

八、`/etc/mtab` 是什么？（超级容易混）

教材明确写了 ⚠注意：

`/etc/mtab` は

`mount / umount` が自動的に管理するファイル

关键对比（一定要背）

文件	作用
<code>/etc/fstab</code>	“应该挂载什么”（静态配置）
<code>/etc/mtab</code>	“现在挂载了什么”（实时状态）

✗ 用户不能手动编辑 `mtab`

九、`umount` 失败的原因（考试常问）

教材最后一个 ⚠ 注意点：

マウントポイント以下を

使用中のユーザーやプログラムがある場合、アンマウントできない

👉 只要有人在用（`cd` 进去、程序打开文件）就不能 `umount`

十、5.3 「マウント / アンマウント」一页考试总结版

你可以直接拿这段当考前速记：

- 文件系统要使用，必须 `mount`
- `mount` 是把文件系统连接到目录树
- `/media` 常用于可移动设备
- `mount` 无参数 → 显示当前挂载状态
- `mount -t ext4 /dev/sdb3 /data`
- `mount -a` → 参考 `/etc/fstab`
- `umount` 可用设备名或挂载点
- `/etc/fstab` 是配置
- `/etc/mtab` 是当前状态
- 使用中的目录无法 `umount`

5.3 文件系统的创建与管理（后半）

逻辑顺序（考试脑内流程）

一、文件系统是「什么时候」被创建的？

📌 不是分区时

📌 而是用 `mkfs` 时

👉 分区只是「切盘」

👉 `mkfs` 才是真正“格式化”

二、mkfs：创建文件系统（超高频）

① 基本概念（考试定义）

mkfs (make filesystem)

👉 在分区或设备上 创建文件系统

⚠ 一旦执行：

👉 原有数据会被破坏

② 基本用法（必背）

`mkfs -t ext4 /dev/sdb3`

拆解理解：

部分	含义
<code>mkfs</code>	创建文件系统
<code>-t ext4</code>	指定文件系统类型
<code>/dev/sdb3</code>	目标分区

📌 考试重点：

- `mkfs` 作用对象是“分区/设备”
 - 不是目录
-

③ 常见等价写法（会考）

`mkfs.ext4 /dev/sdb3`

mkfs.xfs /dev/sdb3

👉 和 `mkfs -t ext4` 效果完全一样

⚠️ 陷阱点：

- `mkfs.ext4` 是命令名的一部分
- 不是参数

三、主要文件系统（5.3 里要你“认识”而不是深挖）

常考的这几个：

文件系统	特点（考试用）
ext4	Linux 默认、通用
XFS	大文件、高性能
VFAT	Windows / U盘
tmpfs	内存文件系统

🔴 tmpfs 特别容易考

四、tmpfs 是什么？（高频陷阱）

tmpfs 的本质：

👉 存在于内存中的文件系统

特点：

- 掉电 → 数据消失
- 不占磁盘
- 常用于 `/dev/shm`

🔴 考试常问：

| tmpfs 是否需要 mkfs?

✅ 不需要

👉 内核自动管理

五、fsck：文件系统检查与修复（必考）

① fsck 是干嘛的？

👉 检查并修复文件系统的一致性

🔴 常用于：

- 非正常关机后
- 文件系统异常时

② 基本用法（考试标准）

fsck /dev/sdb3

或指定类型：

fsck -t ext4 /dev/sdb3

③ 文件系统专用命令（常考）

fsck.ext4 /dev/sdb3

fsck.xfs /dev/sdb3

⚠️ 注意一个大坑（必考）

六、fsck 的超级大坑（LinuC 爱考）

❌ 错误做法：

fsck /dev/sdb3

👉 当该分区正在 mount 中

✅ 正确做法：

- 先 **umount**
- 再 **fsck**

🔴 原因一句话版（考试用）：

挂载中的文件系统进行 fsck

👉 可能导致数据破坏

七、xfs 的特殊点（会单独考）

⚠️ 重点记忆

文件系统	fsck 行为
ext4	fsck 可以修复
XFS	❌ 不能用 fsck 修复

👉 XFS 正确修复命令是：

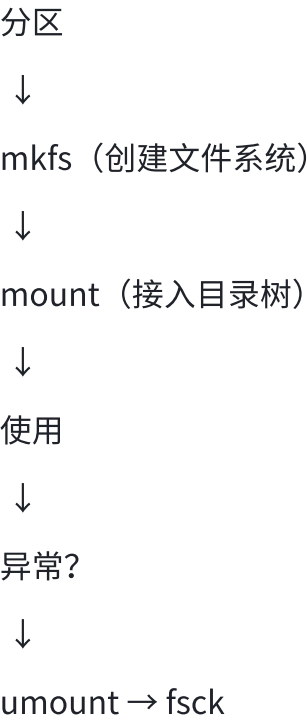
xfs_repair /dev/sdb3

🔴 这道题非常爱考

（尤其是“哪个不能用 fsck 修复”）

八、mkfs / fsck / mount 三者关系（一定要形成链条）

你脑中必须是这条线：



九、5.3 下半场考试速记版（直接背）

- mkfs 用于创建文件系统
- mkfs -t ext4 /dev/sdb3
- mkfs.ext4 与上面等价
- tmpfs 是内存文件系统

- fsck 用于检查修复
- fsck 不能对 mount 中的文件系统执行
- XFS 不能用 fsck 修复
- XFS 使用 xfs_repair

5.2 × 5.3 分区 × 文件系统

高频易错点一次清空

一、分区 vs 文件系统（最基础但最常错）

✗ 易错理解

分区就是文件系统

✓ 正确理解（考试版）

- 分区：磁盘空间的划分（容器）
- 文件系统：在分区里组织数据的规则

📌 一句话记忆

👉 分区 ≠ 文件系统

👉 mkfs 之前，没有文件系统

二、MBR / GPT × 主分区 / 逻辑分区

① MBR 下的限制（必考）

项目	规则
主分区	最多 4 个
扩展分区	占 1 个主分区名额
逻辑分区	建在扩展分区内

📌 陷阱题常问

5 个分区怎么实现？

✓ 3 主 + 1 扩展 + 2 逻辑

② GPT 下的世界

项目	规则
主 / 逻辑	✗ 不区分
分区数	理论上很多
推荐场景	新系统 / 大容量磁盘

📌 一句话

👉 GPT 没有逻辑分区这个概念

三、分区做完以后，下一步是什么？

✗ 常见错误选项

- mount
- fsck

✓ 正确顺序

分区

↓

mkfs（创建文件系统）

↓

mount

📌 考试一眼秒选

四、mkfs vs mount（极高频）

命令	作用对象	本质
mkfs	/dev/sdXN	创建文件系统
mount	设备 → 目录	接入目录树

✗ `mkfs /data`（错）

✓ `mount /dev/sdb3 /data`

五、mount 的两个“身份”

① mount 不带参数

\$ mount

👉 显示当前挂载状态

📌 题目常写：

| mount コマンドをオプションなしで実行すると？

✅ 显示已挂载的文件系统

② mount 省略参数的条件（必考）

mount /data

📌 成立条件：

- /etc/fstab 中 已定义
- 省略的是 设备名或挂载点

❌ 不看 fstab 就能 mount（错）

六、/etc/fstab vs /etc/mtab（经典对比题）

文件	含义
/etc/fstab	静态配置（系统启动、mount 参考）
/etc/mtab	当前挂载状态（自动生成）

📌 考试用一句话

| /etc/mtab 是 mount / umount 使用的文件，不能手动编辑

七、fsck 的致命陷阱（必考）

❌ 错误场景

fsck /dev/sdb3

（该分区正在 mount）

✅ 正确流程

umount /dev/sdb3

fsck /dev/sdb3

✂ 考试关键字

| マウント中のファイルシステム

→ 不能 fsck

八、XFS 的“反直觉点”（非常爱考）

文件系统	fsck 可用?
ext4	✓
xfs	✗

👉 XFS 正确命令：

xfs_repair /dev/sdb3

✂ 出题方式常是：

| fsck が使用できないファイルシステムはどれか？

答案：XFS

九、tmpfs 的三连问（陷阱集中区）

Q1：tmpfs 在哪里？

👉 内存

Q2：是否需要 mkfs？

👉 不需要

Q3：掉电后数据？

👉 全部消失

✂ 常出现选项：

- 占用磁盘空间 ✗
 - 重启后保留 ✗
-

十、mount / umount 的操作对象（易混）

命令	可接收参数
mount	设备名 + 挂载点
umount	设备名 或 挂载点

umount /data

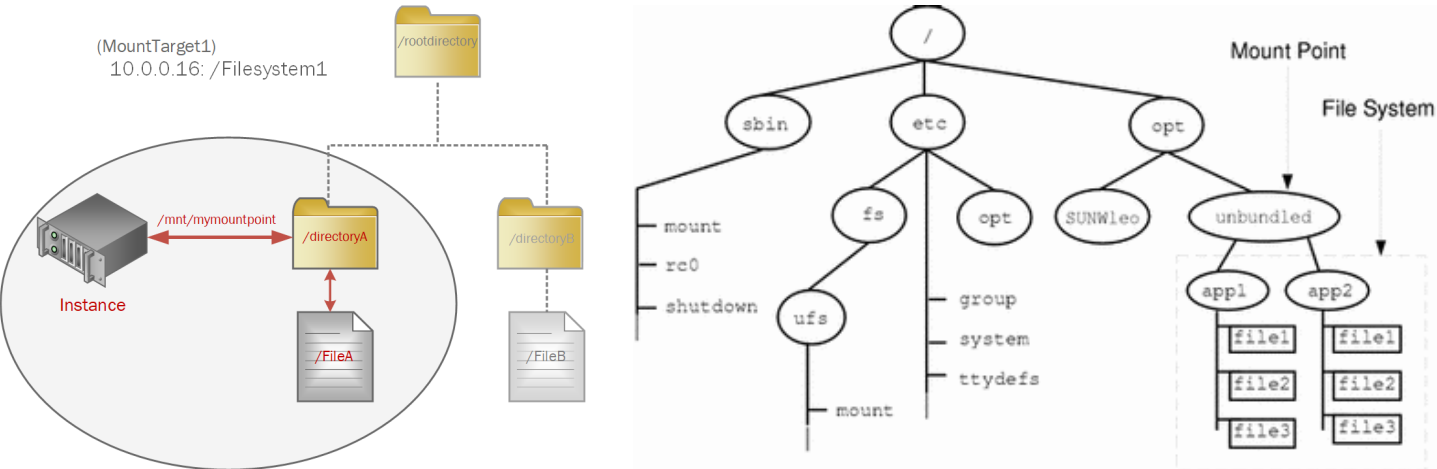
umount /dev/sdb3

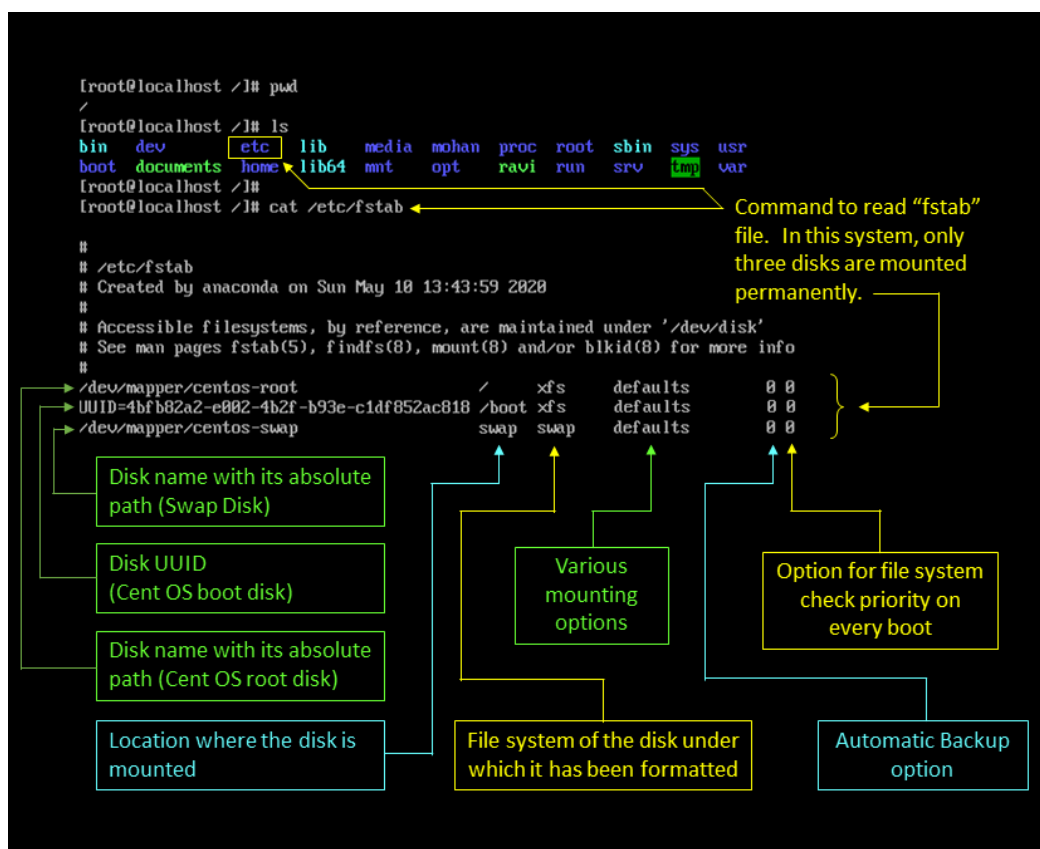
👉 两者等价

十一、考试秒判表（你最后记这个）

看到关键词	直接反应
分区后	mkfs
文件系统作成	mkfs
接入目录	mount
当前挂载状态	mount（无参数）
启动时挂载	/etc/fstab
当前状态文件	/etc/mtab
检查修复	fsck
XFS 修复	xfs_repair
内存文件系统	tmpfs

🧠 mount / umount × fstab / mtab 超清逻辑图（考试视角）





4

下面我把这张图“翻译成成人话 + 考试话”，你照着这个顺序记。

一、四个核心角色（先定位身份）

`/etc/fstab` ← 静态配置文件（规则书）

↓

`mount` ← 执行者

↓

`/etc/mtab` ← 当前状态记录（结果）

`umount` → 更新 `/etc/mtab`

二、每个东西「一句话定性」（超重要）

1 `/etc/fstab`

👉 “我计划要怎么挂载”

- 系统启动时参考

- `mount` 省略参数时参考
- 人工编辑 OK
- 不代表当前是否真的挂载

📌 考试关键词：

起動時 / 自動マウント / 設定ファイル

2 `/etc/mtab`

👉 “现在已经挂了什么”

- `mount / umount` 自动维护
- 不能手动编辑
- 反映当前状态

📌 考试关键词：

現在のマウント状態 / 自動更新

3 `mount`

👉 “照着 `fstab` 或我给的参数去挂”

它会做两件事：

- ① 挂载文件系统
- ② 把结果写入 `/etc/mtab`

📌 关键点（必考）：

- `mount`（无参数）= 显示当前挂载状态
 - `mount /data` → 前提是 `/etc/fstab` 里有 `/data`
-

4 `umount`

👉 “解除挂载 + 更新状态”

- ① 卸载文件系统
- ② 从 `/etc/mtab` 删除记录

📌 重点：

- 不修改 `/etc/fstab`
- 可用 设备名或挂载点

三、完整逻辑流程（你脑中要这样跑）

► 正常启动 or mount -a

系统启动 / mount -a



读取 /etc/fstab



执行 mount



更新 /etc/mtab

► 手动 mount（省略参数）

mount /data

读取 /etc/fstab



找到 /data 对应设备



执行挂载



写入 /etc/mtab

📌 所以这题必考：

| mount /data が成功する条件は？

✅ /etc/fstab に記述がある

► umount

umount /data

解除挂载



更新 /etc/mtab

四、考试最爱“对比陷阱表”

对比点	fstab	mtab
性质	静态	动态
是否人工编辑	✓	✗
启动时使用	✓	✗
mount/ umount 自动更新	✗	✓
当前状态	✗	✓

五、你必须秒答的 5 个句式

我直接给你「看到就反射」的答案模板：

1

| mount コマンドをオプションなしで実行すると？

✓ 当前挂载状态を表示する

2

| /etc/fstab に記述されている内容は？

✓ マウント設定

3

| 現在マウントされているファイルシステムを確認したい

✓ /etc/mtab / mount

4

| umount 実行時に変更されるファイルは？

✓ /etc/mtab

5

| /etc/mtab を手動で編集してもよいか？

✓ いいえ

六、一句话终极记忆法（你背这个就够）

| fstab 是“计划表”，mtab 是“实况转播”

mount 看计划执行，umount 只改实况